Fintech AI Hackathon Starter Guide

Everything you need to build on Alpaca with the Trading API, Broker API & Market Data API

What is Alpaca?

Alpaca is a developer-first brokerage with clean REST + WebSocket APIs for trading, market data, and managing end-user accounts. It powers many of the trading bots, neo-brokers, and investing apps you have seen over the last few years.
Start: alpaca.markets • docs.alpaca.markets • github.com/alpacahq

The three APIs you need to know

API	What it does	Who it's for
Trading API	Place orders, manage one brokerage account, track positions & P&L.	Solo traders, bots, algo strategies.
Broker API	Create & manage many end-user accounts, KYC, funding, commissions.	Neo-brokers, robo-advisors, embedded investing.
Market Data API	Real-time & historical prices for 5,000+ US stocks, 20+ crypto, options.	Any app that needs quotes, bars, or trade feeds.

1. Trading API

Use it when: you want to trade from a single account — yours or the hackathon team's.
Base URLs: `https://paper-api.alpaca.markets/v2` (paper) • `https://api.alpaca.markets/v2` (live)

How to get a Trading API key

1. Sign up at app.alpaca.markets/signup (free, no KYC needed for paper trading).
2. In the dashboard, toggle between Paper and Live in the top-left.
3. Open the **API Keys** panel on the right sidebar and click **Generate New Keys**.
4. Copy the `APCA-API-KEY-ID` and `APCA-API-SECRET-KEY` — the secret is shown only once. Store them as environment variables, never commit them to Git.

What you can build

- **Algo trading bot** — a Python/JS script that reads signals and fires `POST /v2/orders` when conditions match.
- **Portfolio dashboard** — read `/v2/positions`, `/v2/account`, and `/v2/portfolio/history` to show performance over time.
- **Options & crypto strategies** — same endpoints support multi-leg options and 24/7 crypto trading.
- **Backtester** — combine the Market Data API with the Trading API to replay historical bars and simulate strategies.

Docs: Getting Started with Trading API

2. Broker API

Use it when: you are building a product for end-users — they each get their own brokerage account under your Broker API umbrella.
Base URL (sandbox): `https://broker-api.sandbox.alpaca.markets/v1`

How to get a Broker API key

1. Apply for Broker API access at alpaca.markets/broker — for the hackathon, use the **sandbox** which is self-serve and free.
2. Go to the Broker dashboard and switch to the Sandbox environment.
3. Generate a key pair under **API Keys**. Auth uses HTTP Basic (key:secret, base64-encoded).
4. Use the sandbox to create fake end-users, fund them with test money, and place trades — no real funds move.

What you can build

- **Neo-broker / investing app** — onboard users (KYC via the API), open accounts, accept deposits, let them trade.
- **Robo-advisor** — create managed accounts, run rebalancing logic, build thematic or model portfolios.
- **Embedded investing** — add a "Buy this stock" button inside a consumer app, cash-back app, or employer platform.
- **Revenue share** — charge commissions, PFOF, subscriptions, or markup on spreads; Alpaca handles the clearing.

Docs: Getting Started with Broker API • About Broker API

3. Market Data API

Free with every account. Historical + live quotes, trades, and OHLCV bars for 5,000+ US stocks, 20+ crypto pairs, and options. Free tier = IEX feed; paid tier = full SIP real-time.
REST: `https://data.alpaca.markets/v2` • **WebSocket:** `wss://stream.data.alpaca.markets/v2/iex`
Docs: Getting Started with Market Data API

Hello World — your first call (copy-paste)

Install: `pip install alpaca-py` — then run:

```
from alpaca.trading.client import TradingClient
from alpaca.trading.requests import MarketOrderRequest
from alpaca.trading.enums import OrderSide, TimeInForce

client = TradingClient("YOUR_KEY", "YOUR_SECRET", paper=True)
print(client.get_account().cash)  # confirms auth works

client.submit_order(MarketOrderRequest(
    symbol="AAPL", qty=1, side=OrderSide.BUY, time_in_force=TimeInForce.DAY))
```

Stream live quotes (add to any script):

```
from alpaca.data.live import StockDataStream
stream = StockDataStream("YOUR_KEY", "YOUR_SECRET")
async def on_quote(q): print(q.symbol, q.bid_price, q.ask_price)
stream.subscribe_quotes(on_quote, "AAPL", "TSLA"); stream.run()
```

SDKs & GitHub repos

Language	Repo	Notes
Python (recommended)	alpacahq/alpaca-py	Official, actively maintained. Covers Trading, Broker, Market Data.
Node.js / JS	alpacahq/alpaca-trade-api-js	Good for web apps and Next.js backends.

Language	Repo	Notes
Go	alpacahq/alpaca-trade-api-go	High-performance trading loops.
C# / .NET	alpacahq/alpaca-trade-api-csharp	Maintained and typed.
Backtesting	alpacahq/alpaca-backtrader-api	Backtrader integration for strategy research.

Hackathon game plan (90-minute path to a demo)

1. **Pick your angle (5 min).** Trading bot, neo-broker clone, dashboard, or robo-advisor — each maps to one of the APIs above.
2. **Create accounts (10 min).** Sign up for a paper Trading account AND a Broker sandbox account. You will likely want both.
3. **Clone a starter (10 min).** Install `alpaca-py` (`pip install alpaca-py`) or `@alpacahq/alpaca-trade-api` (`npm i @alpacahq/alpaca-trade-api`). Drop your keys into a `.env` file.
4. **Make the first call (5 min).** Hit `GET /v2/account` to confirm auth works. If you see your account JSON, you are good.
5. **Build the happy path (45 min).** One core flow end-to-end: receive a signal → place an order → show the result. Resist the temptation to add features before this works.
6. **Polish + demo script (15 min).** Screen-record a 60-second walkthrough. Judges remember stories, not features.

Gotchas that will cost you an hour each

- **Market hours apply even in paper.** US stock orders outside 9:30-16:00 ET are rejected unless you set `extended_hours=True`. Crypto trades 24/7.
- **Crypto tickers use a slash:** `BTC/USD`, not `BTCUSD`. Options symbols follow OCC format, e.g. `AAPL240119C00150000`.
- **Fractional shares** need `qty` as a float (`qty=0.5`) OR use `notional=100` to buy in dollars. Can't mix both.
- **Rate limit = 200 req/min per key.** Stream quotes with WebSocket instead of polling — it is also what judges expect in a real-time demo.
- **Broker sandbox data is ephemeral** and your paper Trading account is separate — you will have **two** key pairs. Label them in your `.env`.
- **OAuth shortcut:** for neo-broker demos you can skip KYC by letting users log into their own Alpaca account via OAuth. Docs.